

Constellation Detection

CS-GY 6643 · Computer Vision · NYU Tandon

Project 1, Question 5 · Fall 2026

Team size	Up to 2 participants
Submissions	5 per day
Metric	Custom (see Section 5)

Contents

1	Overview	2
2	Description	2
3	Problem Domain	3
4	The challenge is to:	4
5	Evaluation	4
6	Dataset Description	6

1. Overview

Greetings, Traveller.

The sky above you is not the tidy diagram in a textbook. It is nine megapixels of grey static: nebulosity, mosaic seams, instrument scars, and thousands upon thousands of anonymous points of light, each one indistinguishable from the next.

In your hands you carry a fistful of cutouts, small square scraps of sky. Some of them were torn from the very image in front of you. Others were carried here from some other night, some other quarter of the heavens, and belong nowhere in this field at all. They arrive worn from the journey: turned about, resized, softened, speckled with noise. Nothing about a scrap tells you where it came from, or whether it came from here.

Somewhere in that field a figure is drawn. A handful of your scraps are its stars. Others are true stars of this sky that belong to no figure you are looking for, and they will happily pretend otherwise.

Put every scrap back where it belongs. Throw out the ones that never belonged. Then look at what you have recovered, hold it against the fifty-odd charts you carry, and say the constellation's name aloud.

Other astronomers set out tonight on the same errand. The race begins now.

2. Description

The Constellation Detection CS-GY 6643 competition is Project 1 of NYU Tandon's Computer Vision course. You are given a set of **scenes**. Each scene is one large night-sky image together with a folder of small square **query patches**, and your algorithm must decide, for every patch, where it sits in that image or that it does not sit in that image at all, and then name the constellation the scene depicts.

A scene consists of:

- one **sky image** of 3000×3000 pixels, 8-bit grayscale PNG: a dense star field with nebulosity, mosaic seams, imaging artifacts, and thousands of point sources; and
- a folder of **query patches** of 32×32 pixels, 8-bit grayscale PNG.

Every query patch is exactly one of three things, and **you are never told which**:

1. **A figure star.** A star belonging to the constellation figure this scene depicts. It is present in the sky image, and the patch is centred on it.
2. **An off-figure star.** A real star that *is* present in this sky image but is *not* part of the constellation figure: a neighbouring star, a field star, an unrelated source. The patch is centred on it, but it belongs to no part of the answer.
3. **Absent.** A crop of sky that does not occur in this image at all.

Patches of all three kinds were produced the same way and degraded the same way, so **the appearance of a patch tells you nothing about which category it is in**. A patch that is present in the image is not a pristine copy of it. It has been rotated by an arbitrary angle about its own centre, moderately rescaled, shifted by a sub-pixel amount, softened, dimmed or brightened by a smooth illumination drift, and speckled with sensor-style noise and light compression loss. What survives all of this is the one thing you are asked to report: because every geometric change is taken about the patch centre, **the centre of the patch still lands exactly on the point you must report**.

Two further properties of the scenes matter a great deal:

- **You are never handed the whole figure.** Only a subset of a constellation’s stars are issued as query patches.
- **The recovered points are contaminated.** Off-figure stars sit in the same pile as figure stars, and there are usually plenty of them.

So locating every present patch does not, by itself, hand you the constellation. Among the points you recover, you must still work out *which subset of them* forms one of the 48 reference figures, while part of that figure is missing and other genuine stars are actively cluttering the picture.

The `patterns/` folder supplies those 48 candidate figures as line drawings: nodes for stars, edges between them. They are schematics, not photographs. Their pixel size, orientation, handedness and aspect ratio carry no relationship to how the corresponding figure appears in any sky image.

Participants may compete individually or in teams of up to 2 members.

3. Problem Domain

This is a problem astronomers actually have.

Sky surveys produce enormous mosaics stitched from many exposures, taken on different nights, with different instruments, at different orientations and plate scales. Cutouts get separated from their parent frames. Archives hold loose thumbnails whose provenance was lost or whose recorded coordinates are wrong. Before that data is scientifically usable, somebody has to answer two questions: *where did this cutout come from*, and *what am I looking at*. That is the classical problem of plate solving, of registering an unlabelled field against a reference and naming the asterism it contains.

The difficulty is not that stars are hard to see. It is that a star field is enormously self-similar. A 32×32 crop of sky holds a handful of anonymous point sources, and thousands of other places in the same image hold a plausible-looking handful of point sources too. There is no texture, no colour, no semantics, no object to recognise, only geometry. And the geometry you are handed is partial, polluted, and presented under an unknown transformation.

That combination (a weak, repetitive local signal that can only be resolved by a global structural argument) recurs throughout computer vision: in image registration and stitching, in wide-baseline

matching under heavy outlier contamination, in fingerprint and retina matching, in document and logo retrieval, in point-set alignment for medical imaging. The night sky is simply an unusually honest place to study it, because the ground truth is unambiguous and the nuisance factors are physical rather than semantic.

4. The challenge is to:

For **every query patch** in a scene:

1. **Decide whether it occurs in that scene's sky image at all.** If it does not, report `-1`.
2. **If it does occur, localise it.** Report the pixel coordinates (`x`, `y`) of the patch centre in the sky image, where `x` is the column, `y` is the row, and `(0, 0)` is the top-left corner.
3. **Say whether that point belongs to the constellation figure.** Report a third value `m`: `1` if you claim the point is one of the figure's own stars, `0` if it is present in the image but off-figure.

And for **every scene**:

4. **Name the constellation.** One of the 48 figures in `patterns/`, written exactly as its filename stem: lowercase, hyphenated where needed (`canis-major`, `corona-borealis`, `ursa-minor`). Report `unknown` if you decline to guess.

Rules

- **No hardcoding.** No scene-to-name lookup tables, no memorised coordinates, no per-scene patch counts, no keying off filenames or file sizes. Your code must run unchanged on scenes it has never seen, and a leaderboard score your submitted code does not reproduce does not count.
- The hidden scenes may depict constellations that do **not** appear in `train/`. All of them are among the 48 figures in `patterns/`.
- Any approach is permitted: classical, learned, or any combination.
- Alongside your Kaggle submission you must hand in a runnable script.

5. Evaluation

Every scene is scored independently and the per-scene scores are **averaged over scenes**, so a scene with 87 query patches counts exactly as much as a scene with 21.

For a single scene:

$$S = 0.25 \cdot \text{Presence} + 0.20 \cdot \text{Localization} + 0.25 \cdot \text{GeometricRecovery} + 0.30 \cdot \text{Identification}$$

Term	Weight	Measures
Presence	25%	present / absent decision
Localization	20%	pixel accuracy of reported centres
GeometricRecovery	25%	how much of the figure you put on the map
Identification	30%	the constellation name

Presence (25%): the macro-averaged F1 of your present/absent decision over the scene’s real query patches: the F1 of the “present” class and the F1 of the “absent” class, averaged. Declaring everything present, or everything absent, is punished by the class you ignored rather than rewarded by the class you guessed.

Localization (20%): the mean localisation reward over the patches that are **truly present**. A reported point earns full credit within **12 pixels** of the true centre, zero credit at **36 pixels or beyond**, and decays linearly in between. A truly-present patch you declared absent earns nothing here.

GeometricRecovery (25%): how much of the constellation figure you actually put on the map. The figure stars that were issued as query patches are matched **one-to-one** against **all** the points you reported as present, nearest pairs first, under the same 12 px / 36 px reward ramp; the total is divided by the number of those figure stars. Reporting extra off-figure points cannot lower this term, though it does make the constellation harder to name.

Identification (30%): 1 if the constellation name is exactly right, 0 otherwise. This is a **weighted term, not a gate**: getting the name wrong costs you its 30% share, but you keep every point earned for presence, localisation and geometric recovery.

The public leaderboard is computed on roughly 40% of the hidden scenes. Your final standing is computed on the remainder, which you never see a score for until the competition closes. Choose your final submissions on the strength of your method, not on the public score.

5.1 Submission File

One row per scene, and every scene listed in `sample_submission.csv` must appear exactly once. The header is:

```
Id,n_patches,patch_01,patch_02,...,patch_87,constellation
```

- **Id**: the scene folder name, e.g. `constellation_07`. Copy it from the sample submission.
- **n_patches**: how many query patches that scene actually has. Copy it from the sample submission; do not change it. Columns beyond **n_patches** are padding to the widest scene and are ignored.
- **patch_01 ... patch_87**: one cell per query patch, in filename order (`patch_01.png` → column `patch_01`). Each cell is either:
 - `-1`: you claim this patch does not occur in this scene’s sky image; or
 - `(x, y, m)`: the patch centre in image pixels plus your figure-membership claim, e.g. `"(1843, 927, 1)"`.

Cells are parsed leniently: "(x, y, m)", "x y m" and "(x, y)" are all accepted, as is a bare -1. Omitting m is read as 0. Quote any cell containing commas, exactly as the sample submission does.

- **constellation**: a `patterns/` filename stem, lowercase and hyphenated, or **unknown**. One name applies to the whole scene.

Example row:

```
Id,n_patches,patch_01,patch_02,patch_03,...,constellation
constellation_07,24,"(1843, 927, 1)",-1,"(2011, 1502, 0)",...,perseus
```

`sample_submission.csv` is itself a valid, correctly formatted submission in which every patch is declared absent and every scene is **unknown**. It scores above zero, since the absent patches it happens to get right earn Presence credit, and it is the floor you are expected to beat comfortably.

5.2 Scoring

The Kaggle component is worth **65 points**, made up as follows:

Component	Points
Report and 2-minute video explaining your approach	5
Kaggle private leaderboard	40
Beating the baseline on the leaderboard	15
Reaching #1 at any point (one-time)	5
Total	65

- **Report and video (5 points)**. Your Question 5 report, written from the report template, together with a **2-minute video explaining your approach**. Keep the video to two minutes: say what your method does and why, not how the code is organised.
- **Private leaderboard (40 points)**. Your final standing on the private leaderboard, scored with the metric in Section 5.
- **Beating the baseline (15 points)**. Awarded for finishing above the baseline entry on the leaderboard.
- **Reaching #1 (5 points)**. Each student whose team reaches **#1** and beats the baseline at any point earns a one-time **+5** competition points. This includes the final private-leaderboard winner. The bonus is earned at most once per team per competition.

6. Dataset Description

This dataset was curated specifically for this competition. There are three top-level folders (`patterns/`, `train/` and `validation/`) plus two CSVs.

6.1 Files

<code>patterns/</code>	48 reference constellation line-figures
<code><name>_pattern.png</code>	RGBA line drawing: nodes = stars, edges = figure lines; arbitrary size, orientation and aspect ratio
<code>train/<name>/</code>	3 labelled scenes: pisces, scorpius, taurus
<code><name>_image.png</code>	sky image, 3000 x 3000, 8-bit grayscale PNG
<code>patches/patch_XX.png</code>	query patches, 32 x 32, 8-bit grayscale PNG
<code>validation/constellation_XX/</code>	16 unlabelled scenes -- the leaderboard
<code>constellation_XX_image.png</code>	sky image, 3000 x 3000, 8-bit grayscale PNG
<code>patches/patch_XX.png</code>	query patches, 32 x 32, 8-bit grayscale PNG
<code>train_ground_truth.csv</code>	answers for the three train/ scenes
<code>sample_submission.csv</code>	required output format, one row per scene

The three `train/` scenes carry 41, 41 and 34 query patches. The 16 `validation/` scenes carry between 21 and 87 query patches each, 668 in total.

6.2 patterns/

The 48 candidate constellation figures, as RGBA PNG line drawings. Each file is named `<constellation>_pattern.png`, and `<constellation>` is precisely the string the `constellation` column expects. For example `pisces_pattern.png` → `pisces`, `ursa-major_pattern.png` → `ursa-major`.

These are schematic references. Their pixel dimensions, orientation, handedness and aspect ratio carry no information about how the corresponding figure appears in any sky image.

Note also that these 48 are a subset of the 88 IAU-recognised constellations: the answer to every scene is in this folder, but this folder is not the whole sky.

6.3 train/

Three fully labelled scenes, provided as a sanity check, so you can see what real patches, off-figure patches and absent patches look like, and so you can score yourself locally with the same components the leaderboard uses.

They are not part of the leaderboard, and three constellations do not exhaust the 48 possibilities.

6.4 validation/

Sixteen unlabelled scenes. These are the scenes you submit predictions for and the ones the leaderboard scores.

Folder names (`constellation_01`, `constellation_02`, ...) are arbitrary identifiers and reveal nothing about the constellation depicted.

6.5 train_ground_truth.csv

The same layout as the submission file, filled in:

```
Id,n_patches,patch_01,...,patch_87,constellation
```

- **Id**: the train scene name: **pisces**, **scorpius** or **taurus**.
- **n_patches**: the number of real query patches in that scene. Columns past this point are -1 padding and carry no information.
- **patch_XX**: the truth for that query patch:
 - -1: the patch does not occur in this scene’s sky image.
 - (**x**, **y**, **m**): the patch occurs, centred at column **x**, row **y** of the sky image, origin top-left. **m** = 1 means that star is one of the constellation figure’s own stars; **m** = 0 means it is a real star in this image that is not part of the figure.
- **constellation**: the true constellation name for that scene.

6.6 sample_submission.csv

One row for each scored scene, with the correct **Id** and **n_patches** already filled in, every patch cell set to -1, and every **constellation** set to **unknown**.

Reading this file, replacing the cells with your answers and writing it back out is the safest way to guarantee your row set, column count and scene ordering are exactly what the scorer expects.

6.7 Split

19 released scenes: **3 labelled** (**train/**, 16%) and **16 unlabelled** (**validation/**, 84%); or **116** labelled query patches against **668** unlabelled ones, a **15 / 85** split by patch. Every one of the 16 **validation/** scenes is scored; roughly **40%** of them (**6 scenes**) drive the public leaderboard and the remaining **10** decide your final standing. A separate test set of unseen scenes, released to nobody, is what your submitted script is run on to confirm the ranking.

Split	Scenes	Query patches	Share
train/ (labelled)	3	116	16% of scenes
validation/ (leaderboard)	16	668	84% of scenes
public leaderboard	6	—	≈40% of validation
private leaderboard	10	—	≈60% of validation
Held-out test set	unseen; used to confirm the final ranking		